ECE 350: Real-Time Operating Systems Winter 2027

Lecture 22 — Kernel Modules and eBPF

Jeff Zarnett *2026-08-16*

New Functionality

In the introduction, we understood that a modern general-purpose operating system needs to be able to adapt and change. That referenced the idea of supporting new devices (and we'll talk about that in the upcoming device drivers topic). However, it is sometimes desirable to add arbitrary kernel functionality, not just support for new hardware. This could be system calls, yes, but it could also be new functionality.

Modern kernels permit the ability to load and unload pieces of functionality through the use of *modules*, which are exactly what they sound like: self-contained chunks of code that can become part of the kernel. The additional code can be loaded at boot up, or dynamically on request, and unloaded (disabled) if no longer needed.

One increasingly-popular usage of kernel modules is anti-cheating software in online games. Fortnite's own anti-cheating pages[1] say that they use Easy Anti-Cheat's kernel-level protection. Why would a game, of all things, need to load a kernel module to reduce cheating?!

The answer is, as you probably guessed, because the kernel has access to everything in the system. Regardless of how you feel about the ethics of cheating in online multiplayer games, some players are *extremely* dedicated to winning at all costs, through any means. If that's something the game wants to hinder – and they probably do, because competitive games are not very much fun when other players cheat – then they need to use every mechanism available to the would-be cheaters. Fight fire with fire, as it were.

It's tempting to get into all the different ways players can cheat, but that is a bit of a side-track to the topic. It's easy enough for a regular process to detect tampering with the game files; something running as administrator can find out things like whether another process is sending input commands to auto-aim. But users can and would use kernel-level cheats, so the detection needs to be in the kernel as well.

I truly do not know how Fortnite works, but let's imagine a hypothetical example here. Sometimes, games send the player more data than is actually shown on the screen. So your local game client knows the location of a nearby enemy, even if that enemy is behind a wall and is not shown to you as the player. With a kernel-level cheating driver, you can intercept network traffic and use that information to show the info that you are not "supposed" to see – even in a separate window – giving you an advantage because you know where the enemy player is. Detecting that in user mode or as a system administrator would not work; but if the anti-cheat kernel module can examine loaded kernel modules, it could find the existence of something inspecting network traffic.

Kernel modules are specific to a given kernel. This is a part of the reason why it can be difficult to play games on Linux: if the game requires a kernel-level anti-cheating module be loaded, the authors would need to develop and maintain a Linux version of the module. Are they likely to do that? It's not very common... And if they don't, but let players use a Linux version without anti-cheat, the inevitable result is that the cheaters all go over to Linux and use that. From the point of view of players who are not cheating, does it matter what OS the cheaters are using?

Of course, there are numerous other reasons for kernel modules aside from just anti-cheating. Let's move on to the mechanics: how do we create and load them?

[1] `https://www.fortnite.com/competitive/rules-guidelines/rules-library/new-fortnite-anti-cheat-pc-requirements-and-latest-lega region=NAC`

Creating Modules

Writing kernel modules comes with a number of differences compared to what we may be used to. For example, the kernel defines a `linux/printk.h` header that lets us use the `pr_info()` macro to print, something we would normally do with `printf` because the kernel must be able to run without the standard C library [CRKH05]. Similarly, if you need to allocate memory, there are kernel-specific functions like `kmalloc` to accomplish what you want – a regular `malloc` will not work.

Rather like a C program needing a `main()` function, a kernel module has an initialization function that is run on loading the module and a cleanup function that is run to clean up when an unload is taking place, so that the module can be unloaded cleanly; these used to be called `init_module()` and `cleanup_module()` but more modern versions of the kernel permit you to call them anything as long as they are registered with the macros [PSBH00].

The intention of these functions is to do whatever setup and cleanup are necessary for the module to run correctly. This can be no-op if there is nothing to do, but usually there is something. If, for example, memory was allocated in the initialization, deallocate it in the cleanup. Good programming practice would also acknowledge the possibility that loading can fail partway through initialization; if that is the case, undoing anything that's previously done before the failure should take place.

Let's look at the hello-world module from [PSBH00]:

```
#include <linux/init.h> /* Needed for the macros */
#include <linux/module.h> /* Needed by all modules */
#include <linux/printk.h> /* Needed for pr_info() */

static int __init hello_2_init(void)
{
    pr_info("Hello, world 2\n");
    return 0;
}

static void __exit hello_2_exit(void)
{
    pr_info("Goodbye, world 2\n");
}

module_init(hello_2_init);
module_exit(hello_2_exit);

MODULE_LICENSE("GPL");
```

A more complex kernel module must account for concurrency and use concurrency-control constructs like a mutex to ensure that the internal state of the kernel data structures remains consistent. As with memory allocation, there are kernel-specific functions to use in place of the ones in user-space.

System Call Changes. It sounds like we now have a clear pathway to add a system call. In discussing how they worked much earlier on, we said that an integer identifier was put in a predefined location and the trap instruction invoked, which activated the OS. The OS then looks at the identifier and uses that to make a determination of what system call handler to run. So if you want to add a new system call, we just need to bring the new code into the kernel and then update the mapping table so that the new number has a corresponding handler function to run. You might think it's straightforward to replace one, too: bring in the new code and then update the mapping table so the existing number points to the new function instead of the old one.

Except, as collateral damage from a security mitigation, the kernel maintainers broke the ability to add or replace system calls via this table-update approach in version 6.9 [PSBH00]. There is simply no longer such a table in memory that we could alter in x86, and on other architectures it's immutable. That was never a truly supported pathway so breaking it was always a risk. But it did used to work!

If we want to tinker with system calls, then we can of course do that by making the code change and rebuilding the kernel, and rebooting to load it. Just not dynamically with module loading like that. Later on, we'll learn about

how to replace a system call while the kernel is running, when discussing live-patching in the reliability topic.

Upkeep. Earlier, it was mentioned that modules are specific to a given kernel. That means a specific kernel version (e.g., 6.9), not just the Linux kernel in general or even major versions (as in semantic versioning). This is because the kernel itself has no stable internal API. Yes, Linus Torvalds gets extremely mad if a change breaks a user program, but internally the kernel data structures, functions and their signatures, and everything else can change arbitrarily at any time. This is frustrating for developers of modules, and is another reason why game devs do not pursue a Linux anti-cheat module: every kernel update might require major changes to the module.

A helpful kernel post from Greg Kroah-Hartman[2] gives some reasons why the internals are not stable:

1. **Binary Interface.** The compiled binary of the kernel does not remain consistent from build to build for a few reasons:
    - Changes to the C compiler can result in different binary output.
    - Build options can change what `structs` look like or what function implementation blocks appear, and memory alignment.
    - The kernel runs on different architectures (e.g., x86_64, arm).
2. Source Interfaces. The API of the kernel can change too. This is a design decision that gives a high degree of freedom to kernel developers to do things:
    - Fix a bug or improve a design decision without needing to support backwards-compatibility.
    - Retire an old implementation in favour of a new and ostensibly better implementation, also without worrying about backwards-compatibility.
    - Address security issues without having the same constraints about breaking things we discussed in the introduction to the course.

That post gives a specific example about changing the USB interface from synchronous streams to asynchronous ones, permitting increased throughout of USB devices.

The fact that these internal interfaces and data structures can change at any time pushes contributors, sometimes at least, to try to get their code included into the kernel source itself, rather than loaded as a module. If that's the case, any kernel developer making a change or refactor will have to also change your code as well. Otherwise, the kernel won't compile or the tests fail. Still, the standards for contributing to the kernel are very high and things that are very niche are unlikely to be accepted.

You Sure About This?

Should we implement something as a kernel module? It is truly dangerous to do so. The kernel has effectively unlimited ability to do anything in the system and a kernel module that has a vulnerability in it is a target for attackers. While some sort of anti-cheat mechanism is meant to be hard to defeat and is probably designed with security in mind, that is not a given for any module.

Recognizing that modules loaded into the kernel get the highest level of access, knowing what is being loaded is necessary. Linux offers the ability to cryptographically sign modules to verify that they come from the source that they claim to come from. Whether users actually check the signatures or just override and say install anyway is a different matter. It is possible to configure the kernel to reject unsigned modules as a measure of security.

Of course, malicious intent is not necessary for a kernel module to be the root cause of a problem. On July 19, 2024, an update to a kernel module from CrowdStrike – a cybersecurity company of all things – caused a huge number of Windows machines to experience the Blue Screen of Death and be unable to boot up normally. This

[2] `https://docs.kernel.org/process/stable-api-nonsense.html`

was hugely disruptive to numerous industries, including air travel. CrowdStrike published a root cause analysis of what went wrong, if you want to read the details[3]. But there was an out-of-bounds memory read (trying to read element 21 of a 20-element collection). What would normally be a segmentation fault in a regular application... brings down the whole system when it's a kernel module.

Was the CrowdStrike driver signed? Yes! Microsoft also has the ability to sign kernel extensions and the one that caused the incident was signed. There could not be a clearer indication that the signing of a module simply proves where it comes from and is not a guarantee of quality or correctness.

eBPF

If we agree that writing kernel modules is both difficult and risky, an alternative approach is worth pursuing: eBPF. The official documentation says that eBPF is no longer an acronym for anything. The promise of this, according to its documentation, is to extend what the kernel can do without changing the source or loading modules. The hope is that this can be done in a way that permits accomplishing the actual goals without the risk of security or other issues above.

The eBPF approach is to execute a program at designated hooks inside the kernel; each program gets some verification to check for issues [GLP+24]. What are hooks? When the kernel is executing, if it passes a certain point, such as the start of a system call or entry/exit of a function, then that can be the trigger to run the eBPF program. That code runs and then the kernel function executes as normal afterwards.

The eBPF documentation talks about how an eBPF program is written but that's too low-level for us to get into. We write the code using a C-like language and it gets turned into byte code; the byte code is what gets loaded into eBPF. Byte code is an intermediate representation that will be compiled after verification.

Verification is interesting! The eBPF verifier does a static (compile time) analysis of the byte code to determine if it is "safe" to run; if it passes then it goes to a just-in-time compiler and the output is the machine code that actually runs [GLP+24]. The documentation summarizes the verification as saying that it checks three things (1) if the process loading program has the capabilities (as in, privileges) needed to execute it; (2) that the program does not crash or cause any other harm; (3) that the program runs to completion (no infinite loop).

The paper [GLP+24] goes into a lot more of how to actually make that work. The first step is that verifier must build a control-flow graph and ensure that there are no infinite loops or loops whose termination conditions cannot be determined, and that the program always exits normally. Then it tries to do a more exhaustive execution probe of the code, exploring all paths through the code. It also checks for resource usage and permits a program to hold only a single spin-lock at a time, which ensures a deadlock cannot occur (remember why?). The details of verification are much more complex than we want to go into here (even if it's cool), but suffice it to say that an eBPF program goes through this verification process to make sure it is safe to load it.

You would be right to question whether the verification actually demonstrates that the code is safe. Any verification process is also software, and software has bugs. So the verifier can make mistakes too, and may say yes to a program that has a problem or no to one that should be approved. Still, the existence of verification suggests at least a higher chance of staying out of trouble.

The documentation of eBPF compares the utility of this tool as being like Javascript in the browser: it permits a website to be as dynamic and app-like as you want, because Javascript can run arbitrary code, but it's in a sandbox of sorts, and the website can update itself whenever it is ready rather than needing a browser update.

After verification is completed, there's the compilation process. The compilation process is just-in-time (on the fly) and it involves an optimization pass that removes dead or unreachable code before turning it into the binary [GLP+24]. Why would there be dead code? The answer has to do with portability.

[3] https://www.crowdstrike.com/content/dam/crowdstrike/www/en-us/wp/2024/08/Channel-File-291-Incident-Root-Cause-Analysis-08.06.2024.pdf

Portability. The kernel module discussion covers clearly that upkeep of a kernel module is challenging because the kernel internals can change at any time. Programs loaded via eBPF also have this problem, if they look at kernel data structures or use kernel functions. If the name of the structure has changed, for example, the eBPF program will be looking at the wrong thing.

The proposed solution is called eBPF CO-RE (Compile Once, Run Everywhere). Usually, when talking about portability, it means different operating systems (e.g., code that works in Linux and macOS). The cross-OS challenge is one we have avoided by saying that the code examples in the course (and its prerequisite) are meant to run under Linux. Writing an eBPF program that supports different kernel versions is, fortunately, not as complicated as that.

Here's an example of the simple case, where the meaning of a variable has changed and there are simply if-else blocks to handle the differences [Nak20]:

```
extern u32 LINUX_KERNEL_VERSION __kconfig;
extern u32 CONFIG_HZ __kconfig;

u64 utime_ns;

if (LINUX_KERNEL_VERSION >= KERNEL_VERSION(4, 11, 0))
    utime_ns = BPF_CORE_READ(task, utime);
else
    /* convert jiffies to nanoseconds */
    utime_ns = BPF_CORE_READ(task, utime) * (1000000000UL / CONFIG_HZ);
```

We can, of course, extend this approach to the idea of calling a different function based on the kernel version. If the function `work2()` has been introduced in some new kernel version, then we can start using that and use `work1()` in the old version, right? Yes, except that if `work1()` has been removed from the kernel in the current version, the if-else logic still contains a reference to `work1()` and the compiler will not find it and that should fail the compilation. This is why the eBPF approach does the optimization pass to remove dead code: for kernel version x that uses `work2()` and has no `work1()`, the optimizer will see the if-branch referencing `work1()` is dead and remove it, meaning it doesn't appear in the byte code that gets compiled and there is no problem.

A more complicated scenario is where the structures in the kernel themselves have changed. The eBPF CO-RE approach is to have what they call *flavours*. Consider this example [Nak20]:

```
/* up-to-date thread_struct definition matching newer kernels */
struct thread_struct {
    ...
    u64 fsbase;
    ...
};

/* legacy thread_struct definition for <= 4.6 kernels */
struct thread_struct___v46 { /* ___v46 is a "flavor" part */
    ...
    u64 fs;
    ...
};

extern int LINUX_KERNEL_VERSION __kconfig;
...
struct thread_struct *thr = ...;
u64 fsbase;
if (LINUX_KERNEL_VERSION < KERNEL_VERSION(4, 7, 0))
    fsbase = BPF_CORE_READ((struct thread_struct___v46 *)thr, fs);
else
    fsbase = BPF_CORE_READ(thr, fsbase);
```

The source explains that the three underscores in the type name identifies this a flavour of a structure; the flavour part is ignored by the library so that definition will still match to the underlying type when running. The alternative to this flavour approach would be lots of `#ifdef` kind of blocks, which is what we usually see in (allegedly-)portable C code. That would be messy and make it hard to have it be truly compile-once.

There is more complexity to the eBPF CO-RE approach, but we'll stop here.

Game On. This topic opened by looking at anti-cheat kernel modules, so the obvious example of using eBPF is also gaming-related: the `scx_lavd` (Latency-criticality Aware Virtual Deadline) scheduling algorithm[4]. This is a scheduler that is optimized specifically for gaming by improving interactivity, developed specifically for the Steam Deck gaming platform.

This is a possibility because the kernel introduced a new scheduler class (`sched_ext`) that permits a BPF program to take on the role of scheduling for threads in this class. The kernel docs[5] make it clear that "detection of any internal error including stalled runnable tasks aborts the BPF scheduler and reverts all tasks back to the fair-class scheduler". This is another point for safety of using the eBPF approach: if anything goes wrong, the system reverts back to the default scheduler.

The LAVD scheduler balances two goals: improved gaming experience and energy efficiency. Gaming experience is shaped by two things, the average Frames Per Second (FPS) but also "stuttering" [Edg26]. Stuttering is what happens when one of the frame renders is very slow, to the point that it is noticeable to the user; so the scheduler is looking to remove those latency spikes. It's a deadline-based scheduling algorithm, but not a real-time one, and it looks to use what it knows about tasks to figure out which ones are latency critical and assign those shorter deadlines. It uses things like what threads wait for which other ones to figure out which ones are latency-critical.

To actually implement it, a small amount of Rust code is needed: this is because the verifier's strict rules prevent some of the complex calculations. Thus, data is passed back and forth between the eBPF scheduler and the user-space Rust program [Edg26].

Is this a sensible approach? I would argue yes. Getting this scheduler into the kernel source would be a difficult endeavour given that this is a somewhat niche usage and the kernel scheduling algorithm philosophy is to choose something that's best for all workloads. More than that, a scheduler is a high-risk thing to override with a kernel module: if the scheduler gets stuck then the whole system is dead because no recovery actions will be scheduled to execute.

References

[CRKH05] Jonathan Corbet, Alessandro Rubini, and Greg Kroah-Hartman. *Linux Device Drivers*. O'Reilly Media, 3rd edition, 2005.

[Edg26] Jake Edge. Lessons from creating a gaming-oriented scheduler, 2026. Online; accessed 15-August-2026. URL: `https://lwn.net/Articles/1051430/`.

[GLP+24] Bolaji Gbadamosi, Luigi Leonardi, Tobias Pulls, Toke Høiland-Jørgensen, Simone Ferlin-Reiter, Simo Sorce, and Anna Brunström. The ebpf runtime in the linux kernel, 2024. URL: `https://arxiv.org/abs/2410.00026`, `arXiv:2410.00026`.

[Nak20] Andrii Nakryiko. BPF CO-RE (compile once - run everywhere, 2020. Online; accessed 16-August-2026. URL: `https://nakryiko.com/posts/bpf-portability-and-co-re/`.

[PSBH00] Ori Pomerantz, Peter Jay Salzman, Michael Burian, and Jim Huang. *The Linux Kernel Module Programming Guide*. Linux Documentation Project, 2000. Available online at the Linux Documentation Project (TLDP). URL: `https://sysprog21.github.io/lkmpg/`.

[4] `https://sched-ext.com/docs/scheds/rust/scx_lavd`
[5] `https://docs.kernel.org/scheduler/sched-ext.html`